

AI DOCUMENT ANALYSER

Complete System Design & Internal Architecture

Purpose: A MERN-based RAG application where users upload PDF documents and ask natural-language questions. The system extracts and indexes document content, retrieves relevant passages, and uses an LLM to generate grounded answers.

Architecture style: Modular monolith for the MVP, with clear service boundaries that can later evolve into asynchronous workers and specialized infrastructure.

Area	Primary technology / concept
Frontend	React
Backend	Node.js + Express
Database	MongoDB + Mongoose
Document processing	PDF text extraction + optional OCR
Semantic retrieval	Embeddings + MongoDB Vector Search
Generation	LLM API
Authentication	JWT/session-based authentication
Production evolution	Object storage + Redis/BullMQ worker

Design principle: build the smallest architecture that teaches the complete pipeline, then add production infrastructure only when the requirement justifies it.

DOCUMENT MAP

This document is organized from the outside in: what the user sees, how requests move through the system, how documents become searchable, how RAG produces answers, and how the system handles failure and scale.

Section	Coverage
1. System Overview	Purpose, capabilities, core mental model
2. Internal Architecture	Components, boundaries, responsibilities, technology trade-offs
3. User Workflow	Screens, actions, decisions, backend behavior
4. Data Flow	PDF → text → chunks → vectors → retrieval → answer
5. Data Model	Entities, relationships, status lifecycle
6. Request Lifecycle	Internal request/response sequence
7. RAG Internals	Embeddings, vector search, context, LLM
8. Backend Design	Routes, middleware, controllers, services, models
9. API Design	Endpoints and response contracts
10. Security	Authentication, authorization, validation, secrets
11. Edge Cases	Failures and recovery behavior
12. Performance	Async jobs, caching, rate limits, growth path
13. Theory	Reasoning behind major decisions
14. Roadmap	Build order for learning and implementation

1. SYSTEM OVERVIEW

The application is an AI-powered document analysis platform. A user uploads a PDF, the system processes the document into searchable semantic units, and the user can ask questions about that document. Answers are generated from retrieved document context rather than relying only on the model's general knowledge.

Core purpose: turn an unstructured PDF into a searchable knowledge source and provide a conversational interface over that knowledge.

AI Document Analyser — High-Level Architecture

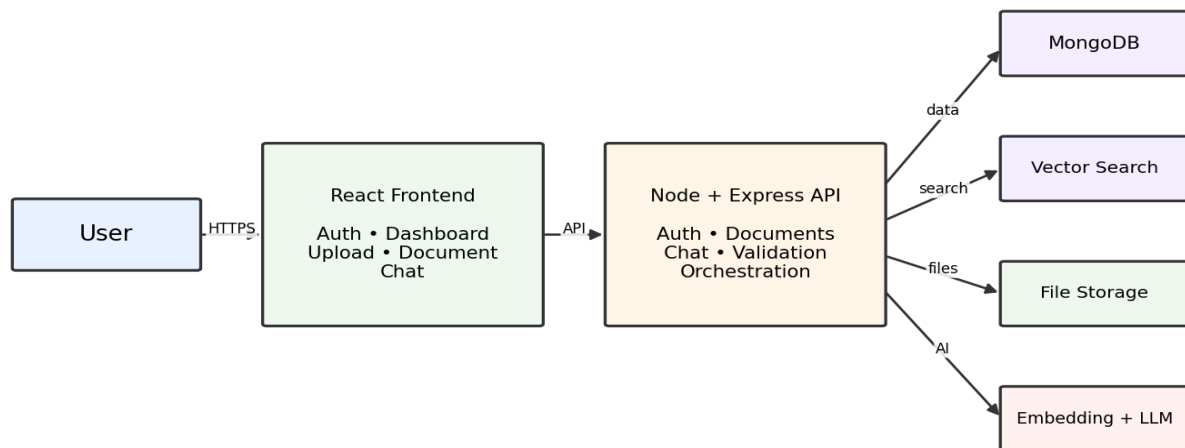


Figure 1 — High-level architecture. React communicates with the trusted backend; database, storage, and AI credentials remain server-side.

The system has four conceptual subsystems:

- **Web application:** React UI for authentication, uploads, document browsing, and chat.
- **Application backend:** Express API for authentication, authorization, validation, business logic, and orchestration.
- **Document/RAG pipeline:** extraction, cleaning, chunking, embeddings, vector retrieval, and context assembly.
- **AI/data infrastructure:** MongoDB, vector search, file storage, embedding model, and LLM provider.

The key mental model is: **ingestion system + retrieval system + generation system**. It is not merely a chatbot.

INGESTION

PDF → text → chunks → embeddings → vector index

RETRIEVAL

question → question embedding → similarity search → relevant chunks

GENERATION

question + relevant chunks → prompt → LLM → answer + sources

2. INTERNAL ARCHITECTURE

The recommended architecture is a **modular monolith** for the first version. One backend deployment contains clearly separated modules instead of splitting the project into microservices prematurely.

This is appropriate for a learning project because it teaches clean boundaries while keeping deployment and debugging manageable.

2.1 Component responsibilities

Component	Responsibility	Why it exists
React	UI, forms, state, document/chat screens	Interactive client experience
Express API	HTTP interface and orchestration	Central trusted application boundary
Auth	Identity and access control	Protect private resources
Document module	Upload, metadata, status, deletion	Own document lifecycle
PDF service	Text extraction/OCR	Convert PDF into usable text
Chunk service	Split text and attach metadata	Precise, bounded retrieval
Embedding service	Text → vector	Semantic representation
Retrieval service	Vector similarity search	Find relevant context
LLM service	Prompt + generation	Natural-language answer
MongoDB	Application state and history	Persistent metadata
Vector Search	Nearest-neighbor retrieval	Semantic matching
File storage	PDF binaries	Separate large files from DB records

AI Document Analyser — High-Level Architecture

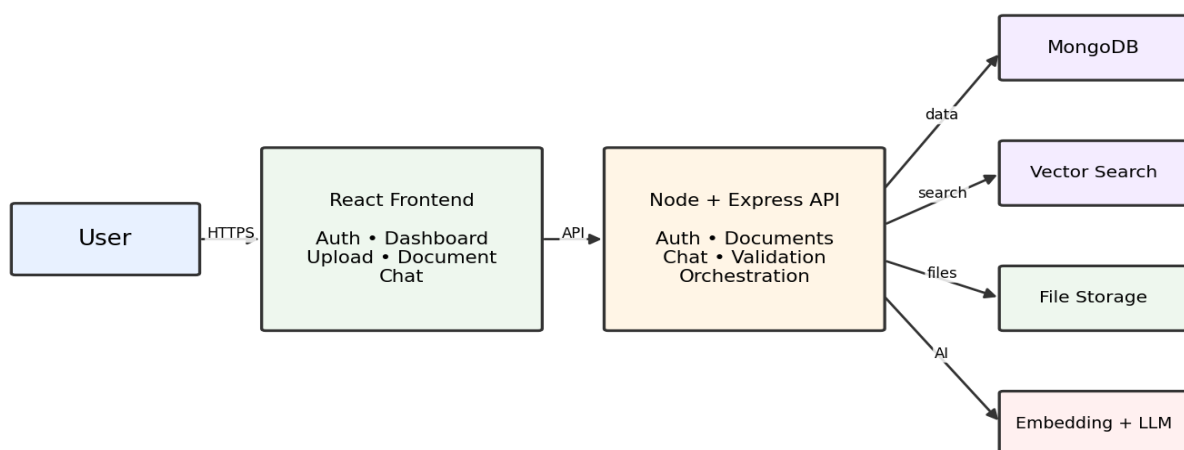


Figure 2 — System component view.

2.2 Technology trade-offs

Decision	Reasoning / trade-off
React	Matches the MERN learning goal and provides a component model. Other frontend frameworks are possible but unnecessary for this scope.
Node + Express	One language across frontend/backend and a transparent request lifecycle. More opinionated frameworks can be introduced later.
MongoDB	Natural MERN fit and flexible metadata; can also support vector search, reducing infrastructure count.
Local file storage initially	Simple to implement and debug; production should use object storage for durability and horizontal scaling.
Synchronous ingestion initially	Simplest mental model. Background jobs become preferable for large files and production workloads.
LLM API	Avoids training/hosting a model. Provider cost and dependency are trade-offs, but implementation complexity is far lower.

3. USER WORKFLOW

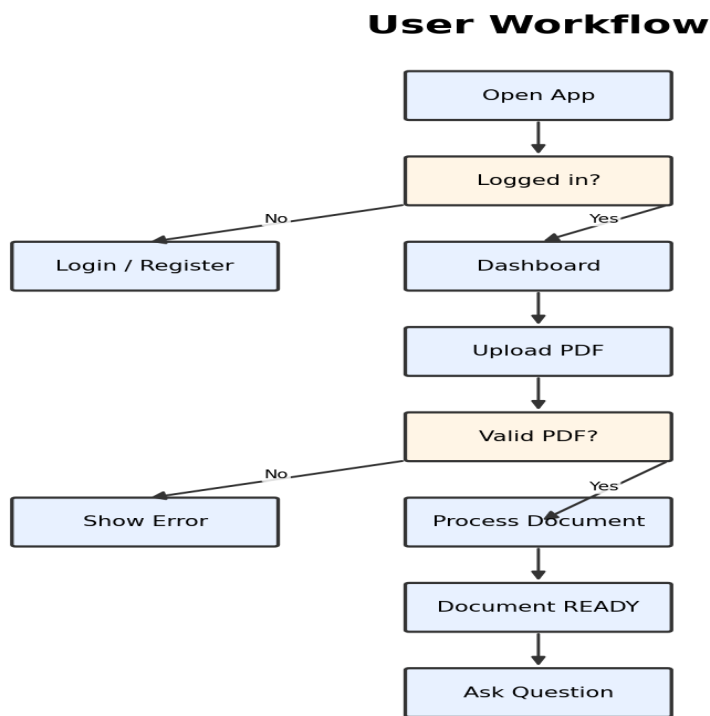


Figure 3 — User workflow including authentication and validation decisions.

3.1 Screen-level workflow

- **Login/Register:** establish identity.
- **Dashboard:** show the user's documents and processing states.
- **Upload:** select and submit a PDF.
- **Processing:** show UPLOADED/PROCESSING/READY/FAILED instead of freezing the UI.
- **Document view:** open one document and its conversation history.
- **Chat:** submit questions and receive answers plus optional sources.

3.2 Behind the scenes

User action	Backend activity
Open app	Load client, inspect auth state, request current user if needed
Login	Validate credentials, create session/token, return identity
Upload PDF	Authenticate, validate file, store binary, create document record
Processing	Extract, clean, chunk, embed, index, update status
Open document	Verify ownership and return metadata/conversations
Ask question	Authenticate, verify ownership, retrieve context, call LLM, save messages
Delete	Authorize, remove file and associated data according to retention policy

4. DATA FLOW

Document Ingestion Pipeline

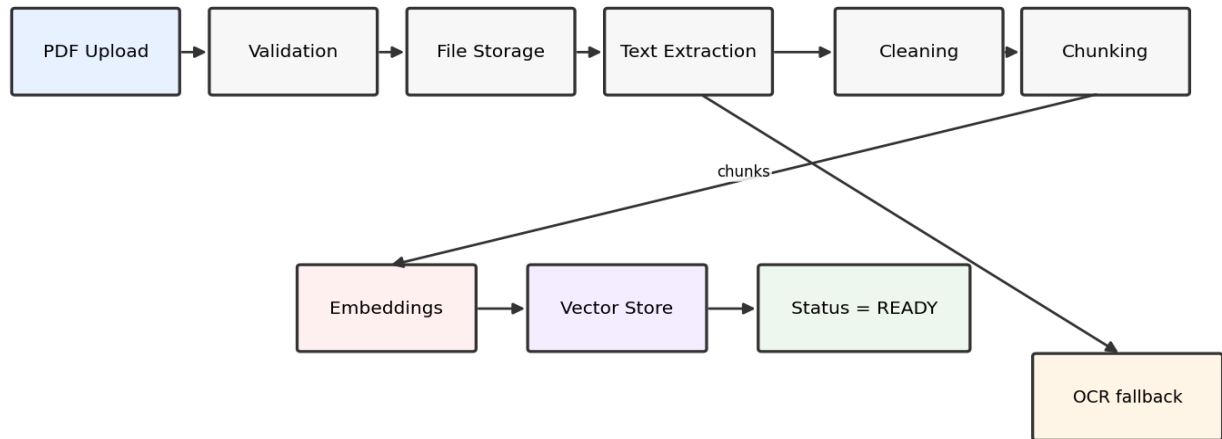


Figure 4 — Document ingestion data flow.

4.1 Upload data

```
multipart/form-data
file = research-paper.pdf

Validation:
- authenticated user?
- file present?
- allowed type?
- size within limit?
- processable PDF?

Document metadata:
userId, filename, storageKey, mimeType,
fileSize, status, createdAt
```

4.2 Text transformation

```
PDF binary
↓
Extracted text
↓
Normalized / cleaned text
↓
Chunk 1, Chunk 2, ... Chunk N
↓
Embedding vector for each chunk
↓
Vector index + metadata
```

4.3 Why metadata matters

- **pageNumber:** source citation and UI navigation.
- **chunkIndex:** ordering and debugging.
- **documentId:** prevents retrieval across unrelated documents.

- **userId:** authorization boundary.
- **status:** processing lifecycle.

RAG Question-Answering Pipeline

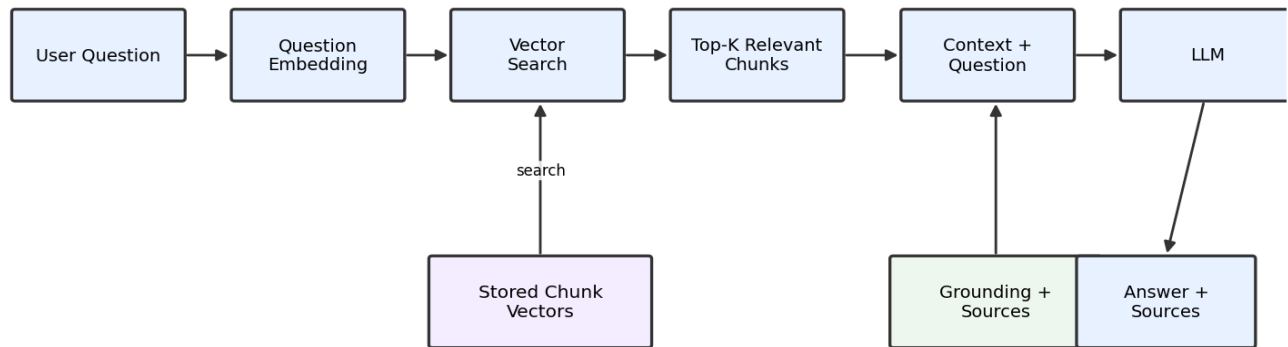


Figure 5 — RAG data flow from question to answer.

5. DATABASE & DATA MODEL

Logical ER / Data Model

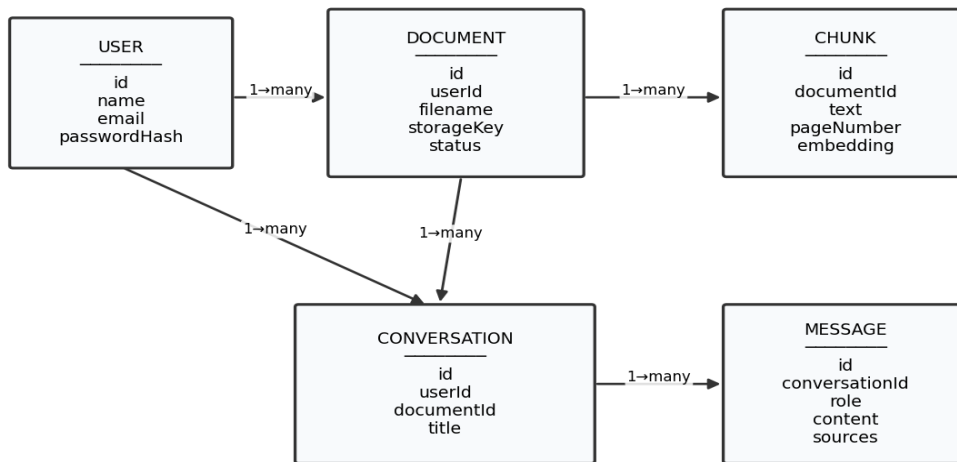


Figure 6 — Logical entity relationship model.

Entity	Important fields	Purpose
User	<code>_id</code> , name, email, passwordHash, createdAt	Identity and ownership
Document	<code>_id</code> , userId, filename, storageKey, mimeType, fileSize, status, pageCount	PDF metadata and lifecycle
Chunk	<code>_id</code> , documentId, text, pageNumber, chunkIndex, embedding	Searchable semantic unit
Conversation	<code>_id</code> , userId, documentId, title, createdAt	Chat session
Message	<code>_id</code> , conversationId, role, content, sources, createdAt	Conversation history/citations

5.1 Relationships

- One user owns many documents.
- One document contains many chunks.
- One document can have many conversations.
- One conversation contains many messages.
- Every private document lookup is constrained by authenticated user identity.

5.2 Document state machine

UPLOADED → PROCESSING → EXTRACTING → CHUNKING → EMBEDDING → READY
■ ERROR / FAILED

Explicit state is important because document processing is not instantaneous. Both the backend and frontend need to know whether a document is usable.

6. REQUEST / RESPONSE & PROCESS LIFECYCLE

Question Request Sequence

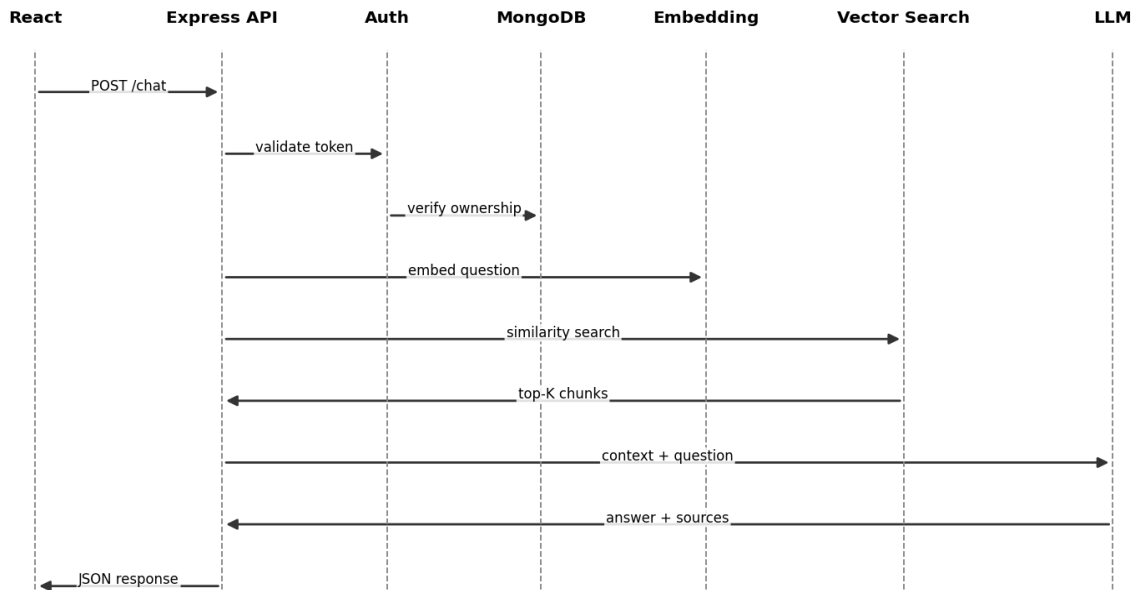


Figure 7 — Question request sequence.

6.1 Layered request lifecycle

```
HTTP Request
↓
Router
↓
Authentication middleware
↓
Validation middleware
↓
Controller
↓
Business service
  ■■■ database
  ■■■ vector retrieval
  ■■■ LLM provider
↓
Controller
↓
HTTP Response
```

6.2 Why layers matter

- **Routes:** public API contract, not business logic.
- **Middleware:** cross-cutting concerns such as auth and validation.
- **Controllers:** translate HTTP into application operations.
- **Services:** reusable business logic and orchestration.
- **Models/repositories:** persistence concerns.
- **AI adapters:** isolate provider-specific SDK/API behavior.

6.3 Example chat request

```
POST /api/documents/:documentId/chat
```

```
{ "question": "What is the main conclusion?" }
```

- authenticate
- verify ownership
- embed question
- vector search
- assemble context
- call LLM
- save messages
- return answer + sources

7. RAG INTERNALS & THEORY

RAG means Retrieval-Augmented Generation. It separates the task into finding relevant information and generating a response from that information.

RAG Question-Answering Pipeline

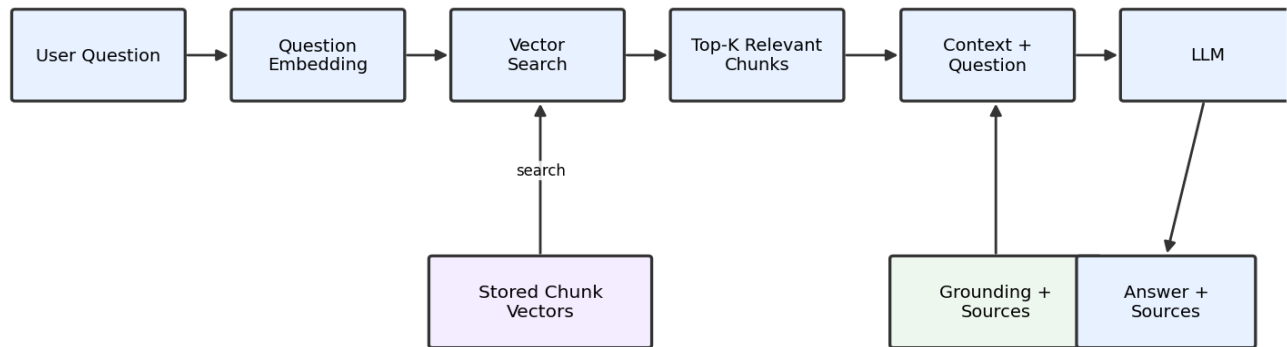


Figure 8 — RAG architecture.

7.1 Embeddings

An embedding is a numerical representation of text that captures semantic relationships. Similar meanings tend to map to nearby vectors.

```
"car" → [0.21, 0.53, -0.11, ...]
"automobile" → [0.20, 0.51, -0.10, ...]
```

The system creates embeddings for document chunks during ingestion and for user questions during retrieval.

7.2 Similarity search

```
Question vector
↓
Compare with chunk vectors
↓
Chunk 1 → 0.31
Chunk 2 → 0.92
Chunk 3 → 0.18
Chunk 4 → 0.87
↓
Top-K → Chunk 2, Chunk 4, ...
```

Cosine similarity is a common vector similarity measure. The vector search engine performs the nearest-neighbor work; the application controls filters, top-K, and context selection.

7.3 Context assembly and grounding

The application should provide a bounded set of relevant chunks. Sending the entire document to the LLM is inefficient and can reduce answer quality.

```
System:
Answer using the supplied document context.
```

If the context is insufficient, say so.

Context:
[retrieved chunks]

Question:
[user question]

8. BACKEND INTERNAL DESIGN

Backend Internal Layering

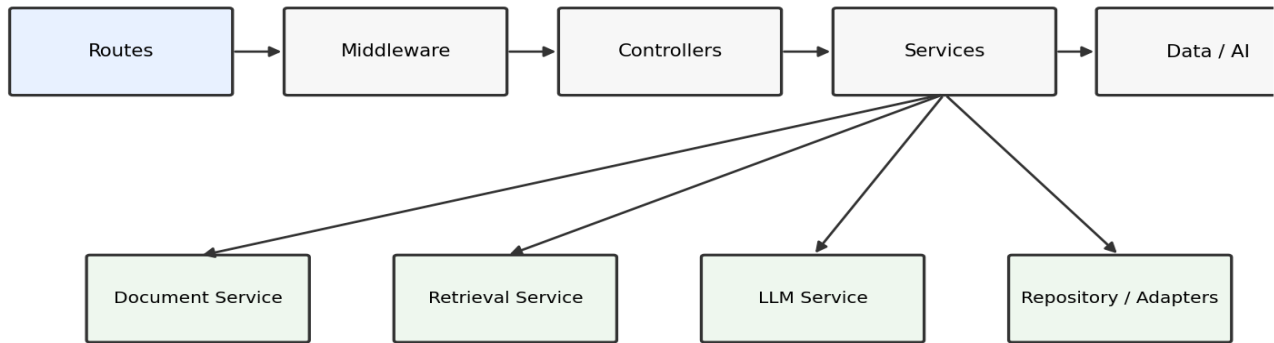


Figure 9 — Backend layering.

8.1 Recommended project structure

```
server/  
  ■■■ src/  
  ■■■ config/  
  ■ ■■■ database.js  
  ■ ■■■ environment.js  
  ■■■ models/  
  ■ ■■■ User.js  
  ■ ■■■ Document.js  
  ■ ■■■ Chunk.js  
  ■ ■■■ Conversation.js  
  ■ ■■■ Message.js  
  ■■■ routes/  
  ■ ■■■ auth.routes.js  
  ■ ■■■ document.routes.js  
  ■ ■■■ chat.routes.js  
  ■■■ controllers/  
  ■ ■■■ auth.controller.js  
  ■ ■■■ document.controller.js  
  ■ ■■■ chat.controller.js  
  ■■■ services/  
  ■ ■■■ pdf.service.js  
  ■ ■■■ chunk.service.js  
  ■ ■■■ embedding.service.js  
  ■ ■■■ retrieval.service.js  
  ■ ■■■ llm.service.js  
  ■ ■■■ document.service.js  
  ■■■ middleware/  
  ■ ■■■ auth.middleware.js  
  ■ ■■■ upload.middleware.js  
  ■ ■■■ error.middleware.js  
  ■■■ server.js
```

8.2 Example call graph

```
POST /api/documents/:id/chat  
↓
```

```
chat.controller
↓
chat.service
  ■■■ authorization
  ■■■ embedding.service
  ■■■ retrieval.service
  ■■■ llm.service
  ■■■ message persistence
↓
JSON response
```

This structure prevents the common failure mode of placing authentication, database queries, PDF logic, vector search, and AI calls into one huge route handler.

9. API DESIGN

Area	Method	Endpoint	Purpose
Auth	POST	/api/auth/register	Create account
Auth	POST	/api/auth/login	Authenticate
Auth	GET	/api/auth/me	Current user
Documents	POST	/api/documents	Upload PDF
Documents	GET	/api/documents	List user's documents
Documents	GET	/api/documents/:id	Get document
Documents	GET	/api/documents/:id/status	Get processing state
Documents	DELETE	/api/documents/:id	Delete document
Chat	POST	/api/documents/:id/chat	Ask question
Chat	GET	/api/documents/:id/conversations	List conversations
Chat	GET	/api/conversations/:id/messages	Get messages

9.1 Consistent response contract

```
Success:
{
  "success": true,
  "data": { ... }
}

Error:
{
  "success": false,
  "error": {
    "code": "DOCUMENT_NOT_FOUND",
    "message": "Document not found"
  }
}
```

A stable response shape simplifies frontend error handling and makes the API easier to test and document.

10. SECURITY DESIGN

Security is a backend responsibility and should not depend on the React client behaving correctly.

Concept	Question
Authentication	Who is the user?
Authorization	Is this user allowed to access this resource?

10.1 Critical rules

- Never trust a document ID from the browser; verify ownership on the server.
- Never expose LLM/API keys to React.
- Hash passwords; never store plaintext passwords.
- Validate file type, size, and processing safety.
- Use HTTPS in production.
- Rate-limit expensive upload/chat endpoints.
- Do not return stack traces or secrets to clients.
- Use environment variables or a secret manager for credentials.

10.2 Ownership check

```
authenticatedUserId = req.user.id
document = findDocument(documentId)

if document.userId !== authenticatedUserId:
    reject with 403
```

The browser is not a trusted security boundary. Authorization must happen on the server.

11. OUTCOMES, EDGE CASES & FAILURE HANDLING

Failure Handling Strategy

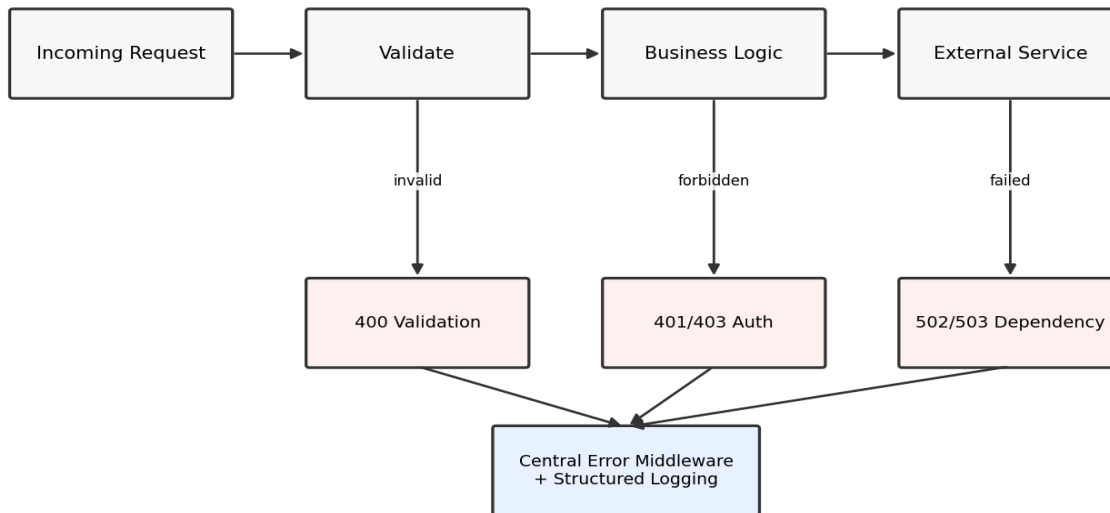


Figure 10 — Failure handling strategy.

Scenario	Expected behavior
Non-PDF upload	Reject with 400 validation error
Oversized PDF	Reject before expensive processing
Corrupted PDF	Set status to FAILED and show error/retry
Scanned PDF	Use OCR if normal text extraction is insufficient
No relevant answer	Say the answer cannot be determined from the document
LLM timeout	Return controlled dependency error and log it
Embedding failure	Keep processing/failed state and allow retry
Unauthorized access	Return authorization-safe error
Too many requests	Rate-limit before expensive AI calls
Duplicate upload	Optionally detect by file hash and reuse or store explicitly

11.1 User-visible state

```
research.pdf READY
notes.pdf PROCESSING
broken.pdf FAILED

User:
What is the main conclusion?

AI:
The main conclusion is ...

Sources:
Page 12
Page 13
```

12. PERFORMANCE & SCALABILITY

The first version should be intentionally simple. The architecture should nevertheless leave clear paths for scaling.

12.1 Async document processing

```
Synchronous MVP:  
Upload → Process → Respond  
  
Production:  
Upload → Store → Queue Job → Respond 202  
↓  
Worker  
↓  
Extract → Chunk → Embed  
↓  
READY
```

A background worker prevents long processing jobs from occupying normal HTTP request lifetimes.

12.2 Useful optimizations

- Bound retrieval with top-K instead of returning every matching chunk.
- Use MongoDB indexes for frequent queries.
- Cache repeated expensive operations where appropriate.
- Store extracted text/chunks so the PDF is not reparsed for every question.
- Keep prompts bounded to control latency and cost.
- Rate-limit expensive endpoints.
- Use object storage when horizontally scaling.

12.3 Growth path

```
Stage 1: React + Express + MongoDB + local uploads + AI API  
Stage 2: Object storage + vector indexes + structured logging  
Stage 3: Redis/BullMQ + workers + retry policies  
Stage 4: Separate heavy processing services only if workload justifies them
```

13. THEORY & DESIGN REASONING

Decision	Reasoning
Modular monolith first	Simple deployment/debugging while preserving clean internal boundaries.
Separate controllers/services	Keeps HTTP concerns out of business logic.
PDF outside normal DB records	Large binaries are better handled by file/object storage.
Chunk before embedding	Improves retrieval precision and avoids oversized contexts.
Embed documents and questions	Allows semantic matching rather than exact keyword matching.
Retrieve before generating	Grounds the LLM in document evidence.
Store source metadata	Provides traceability and user verification.
Explicit document status	Makes long-running processing observable.
Async workers later	Prevents long jobs from blocking HTTP.
Provider adapters	Makes AI provider changes less invasive.

The general architectural rule is: **separate responsibilities according to why they change**. UI concerns, business rules, persistence, and external AI integrations should not be tightly coupled.

14. IMPLEMENTATION ROADMAP

Phase	Build	Concepts learned
1	Node + Express	HTTP, routing, middleware
2	MongoDB + Mongoose	Schemas, models, CRUD
3	React ↔ API	Client/server communication
4	Authentication	JWT/session, hashing, protected routes
5	PDF upload	multipart/form-data, file middleware
6	PDF extraction	Document processing
7	Chunking	Text segmentation and metadata
8	Embeddings	Vectors and semantic representation
9	Vector search	Similarity retrieval
10	LLM integration	Prompt and context generation
11	RAG chat	Retrieval + generation
12	Conversation history	Message persistence
13	Security hardening	Authorization, limits, validation
14	Async workers	Queues, retries, background processing
15	Deployment	Production configuration and observability

Build and test each phase before adding the next. Do not begin by installing every AI package. Start with the Express/MongoDB foundation and introduce one capability at a time.

Recommended learning loop

1. Understand the concept
2. Implement the smallest version
3. Test it manually
4. Inspect request/response behavior
5. Add validation and error handling
6. Refactor into the correct module/service
7. Continue to the next subsystem

15. COMPLETE END-TO-END EXAMPLE

Example: the user uploads **Machine_Learning.pdf** and asks “*What is supervised learning?*”.

Upload phase

```
Machine_Learning.pdf
↓
POST /api/documents
↓
Authenticate + validate
↓
Store file
↓
Document(status=PROCESSING)
↓
Extract text
↓
Chunk
↓
Generate embeddings
↓
Store vectors
↓
Document(status=READY)
```

Question phase

```
User question
↓
Question embedding
↓
Vector search constrained to this document
↓
Top relevant chunks
↓
Context + question
↓
LLM
↓
Answer + source pages
↓
Save user/assistant messages
↓
React displays result
```

This single example crosses every major boundary: authentication, authorization, persistence, document processing, semantic retrieval, AI generation, and user-facing output.

16. FINAL ARCHITECTURE SUMMARY

AI Document Analyser — High-Level Architecture

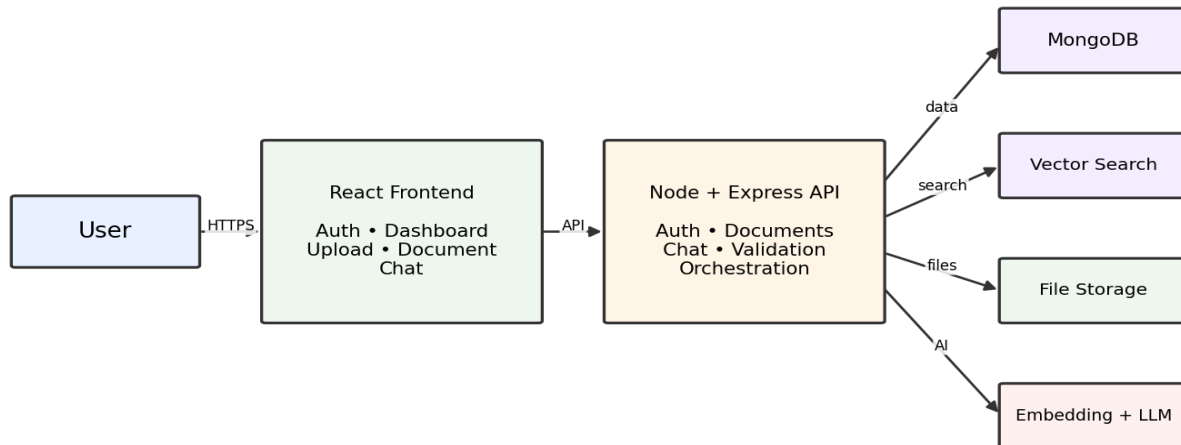


Figure 11 — Final system overview.

Pipeline	Flow
Document ingestion	PDF → validation → storage → extraction → chunking → embeddings → vector index → READY
Question answering	Question → authentication → ownership check → question embedding → vector search → context → LLM → answer + sources

The application is deliberately designed as a modular monolith first. Heavy processing can later move to workers; file storage can move to object storage; AI providers can remain isolated behind adapters; and vector search can be replaced if scale or product requirements demand it.

The deepest concept to retain

The LLM is the generation layer, not the document database. The application's responsibility is to retrieve relevant context from the user's document and give that context to the LLM in a controlled prompt. This is the core architectural idea behind the document analyser.

Final mental model

```
DOCUMENT
↓
INGEST
↓
INDEX
↓
RETRIEVE
↓
GENERATE
↓
ANSWER

PDF → Text → Chunks → Vectors
↓
Question → Vector Search → Context → LLM → Answer
```

End of system design document.